Ribhus - Explained Like You're Learning It From Scratch

For: you, as a student of this codebase

Goal: after reading this, you should be able to point at any folder and say what it is for, when it runs, and what calls what

Version covered: Ribhus 0.1.0 (what exists today)

Tip: keep this open next to the repo. When a sentence names a file, open that file once. Seeing the real code after the story is how it sticks.

Part 0 - Start here: what problem does Ribhus solve?

A normal chatbot only talks.

You say: "Find a hotel in Goa under Rs8000 and WhatsApp three options to Rahul."

A chatbot replies with text. You still have to search, copy, open WhatsApp, paste, send.

Ribhus is built so the AI can do the steps (search, browse, draft a message) and only stop when something is risky (like actually pressing Send).

So Ribhus = chat + hands.

Those "hands" are called tools.

Part 1 - The three ideas you must lock in first

If you remember only three ideas from this whole document, make them these.

Idea 1: The brain is a rented model

Ribhus does not train its own big neural net for chatting.

It calls an LLM API (OpenAI, OpenRouter, or a local Ollama server).
That model can:

- write text replies, and
- ask to call tools by name with JSON arguments.

Ribhus's job is to host that loop: feed the model context, run the tools it asks for, feed results back, repeat.

File that talks to the model: src/ribhus/models/openai_compatible.py
Class: OpenAICompatibleModel

Analogy: the model is a smart intern on Zoom. Ribhus is the office that gives the intern a phone, a browser, and a rulebook ("ask before sending money").

Idea 2: Tools are real functions with a name

A tool is just a Python class with:

1. a name the model can say (web_search, whatsapp_send, ...)
2. a JSON schema describing arguments
3. an async execute(...) method that returns a string observation

File: src/ribhus/tools/base.py

```
class Tool(ABC):
    name: str
    description: str
    risk_level: str = "low"
    force_approval: bool = False  # if True, never auto-skip human approval

    def input_schema(self) -> dict: ...
    async def execute(self, arguments: dict) -> str: ...
```

When the model says "call web_search with query=...", Ribhus runs that Python code and pastes the result back into the conversation as a tool message.

Idea 3: Dangerous tools must ask a human

Reading a webpage is usually fine.

Sending a WhatsApp message is not.

Ribhus attaches a risk level to each tool and a PolicyEngine that can pause and ask you: Approve / Edit / Cancel.

Files: src/ribhus/policy/engine.py, src/ribhus/policy/levels.py

You will see this pattern everywhere: CLI panels, dashboard buttons, WhatsApp Note-to-Self replies - same idea, different UI.

Part 2 - Where does Ribhus live on disk?

The code (the program)

```
Ribhus/                          ? git repo
  src/ribhus/                    ? the Python package
  web/                           ? React dashboard UI
  skills/                        ? skill packages you can install
  tests/                         ? pytest
  docs/                          ? guides (this file)
  pyproject.toml                 ? dependencies + `ribhus` CLI entry
```

When you type ribhus in the terminal, Python runs:

```
ribhus.cli.main:run
```

That is declared in pyproject.toml under [project.scripts].

The data (your machine's memory of Ribhus)

While the code is in the repo, your personal state is stored in a hidden folder:

```
<your-workspace>/.ribhus/
```

Examples:

File / folder	What a student should think
memory.sqlite	"Things Ribhus remembered about me"
tasks.json	"Dashboard runs / history of agent turns"
bots.json	"Named bots I created"
chats.json	"DM and group chat transcripts"

whatsapp_profile/	"Logged-in WhatsApp Web browser profile"
gmail_profile/	"Logged-in Gmail browser profile"
audit.jsonl	"Who approved what tool call"
mcp.json	"External tool servers to connect"

.ribhus/ is gitignored on purpose. It is your data, not the project's source.

Config that controls behavior: environment variables with prefix RIBHUS_, usually in a .env file.
Reader class: Settings in src/ribhus/config.py.

Part 3 - The heartbeat: the Agent loop

This is the most important algorithm in the whole project.

File: src/ribhus/agent/loop.py
Class: Agent
Method: async def run(prompt, history=None)

Plain English

1. Build a big starting message list (system rules + memory + skills + user prompt).
2. Ask the model: "What next?"
3. If the model answers with normal text -> you're done. Show that text.
4. If the model answers with tool calls -> run each tool, append results, go back to step 2.
5. Stop if finished, or if you hit max_iterations (default comes from Settings, often 15).

Why a loop?

Because one model call is rarely enough.

Example:

- Call 1: model asks for web_search
- Call 2: model asks for web_extract on the best URL
- Call 3: model writes the final answer

That is three "iterations."

What is inside state.messages?

It is a chat transcript the model understands:

role	meaning
system	Standing rules ("You are Ribhus...")
user	What you typed (and sometimes nudges Ribhus injects)
assistant	Model text and/or tool_call requests
tool	String result of a tool, tied to a tool_call id

File for system text: src/ribhus/agent/state.py -> SYSTEM_PROMPT
Class for the bag of messages: AgentState

Tiny mental picture

```
You: "Research Zoho"
        |
        v
   [Agent.run]
        |
        +-? model: "please call web_search"
        |         |
        |         v
        |      ToolRegistry.execute("web_search", {...})
        |         |
        |         v
        |      result string goes into messages
        |
        +-? model: "please call web_extract"
        |
        +-? model: "Here is the brief..."  ? final_text
```

Extra brains inside the loop (nudges)

The loop is not only "run tools blindly." It has helper detectors, because models misbehave:

Helper	What it catches
prefers_whatsapp_channel	"Message Rahul" -> don't use Gmail
looks_whatsapp_read_only	"Summarize my chat" -> don't send
asks_send_approval_in_chat	Model says "please approve?" in text instead of calling send
looks_multi_step + todo nudges	Multi-step work should make a checklist

These live in loop.py and plan_loop.py. Think of them as a teaching assistant sitting next to the model saying "wrong tool; try again."

Part 4 - Tools in depth (the hands)

How a tool gets registered

File: src/ribhus/tools/registry.py

1. Create a Tool subclass.
2. Call registry.register(tool).
3. default_registry(...) builds the usual set for CLI/dashboard.

When the agent starts, it sends schemas (names + JSON parameters) to the model so the model knows what it is allowed to ask for.

What happens on registry.execute(name, args) - step by step

Open this function and read it slowly. It is the security checkpoint.

```
1. Find the Tool by name
2. Compute effective risk (some tools change risk by arguments)
3. preflight(args)
      -> if it returns an error string, STOP (no approval UI, no execute)
4. policy.authorize(tool, args)
      -> may ask the human
      -> if denied, return denial string to the model
5. tool.execute(args)
      -> real work happens
6. write an audit log line
7. return the result string to the model
```

Important student insight: the model never receives a Python exception object for a normal denial. It receives a string like "User denied high-risk tool...". Then it has to keep talking based on that.

Catalog of built-in tools (what each family is for)

Internet research

Tool	Everyday meaning
web_search	"Google-ish search" via DuckDuckGo library (ddgs)
web_extract	"Open this URL and give me the article text"
http_request	"Call an API / raw HTTP"

Files & code on your machine

Tool	Everyday meaning
read_file	Read a file under the workspace
write_file / edit_file	Create or patch files (higher risk)
python	Run a short Python snippet, capture prints
terminal	Run a short shell command
process	Start/watch/kill a long-running process

Workspace sandbox: file tools are meant to stay inside your project folder, not roam the whole disk.

Browser (generic websites)

Tool	Everyday meaning
browser	Control Chromium: open page, snapshot elements as @e3, click, type, screenshot
vision	Ask a vision model "what's in this screenshot?"

Critical rule: WhatsApp and Gmail are not supposed to be driven by the generic browser tool. They have their own channel sessions (Part 8). Why? Different login cookies, different safety rules, different rate limits.

Places around you

Tool	Everyday meaning
location	"Where am I?" / nearby restaurants (device GPS on macOS, else IP, else saved)
maps	Build Google Maps place/directions links

Talking to the human mid-run

Tool	Everyday meaning
clarify	Ask you a precise question (area of Goa? dates?) before guessing
todo	Make a checklist for multi-step work

Memory tools (if memory is enabled)

memory_remember, search/forget helpers - so the model can save a fact on purpose, not only via automatic extraction.

Messaging channels

WhatsApp tools (whatsapp_open_chat, whatsapp_reply, whatsapp_send, ...)
Gmail tools (email_list, email_send, ...)

whatsapp_send / email_send set force_approval = True. That means even if you set RIBHUS_AUTO_APPROVE=true, Ribhus still must ask before sending. This is intentional.

Bot-to-bot

Tool	Everyday meaning
message_bot	"Pass this note/work to another named bot" (async)

MCP tools

If you connect an MCP server named github, a tool might appear as github__list_issues. Same checkpoint as built-in tools.

Part 5 - Policy: the permission system

Risk levels

Think of a ladder:

```
low  ->  medium  ->  high  ->  critical
```

Settings say: "Ask me starting at high" (RIBHUS_APPROVE_AT=high).

So:

- low tools usually run quietly
- high/critical tools pause for approval
- tools with force_approval=True always pause

Who is the "approver"?

PolicyEngine does not draw UI itself. It calls a function you injected:

```
PolicyEngine(approver=some_callable)
```

That callable is different depending on where you started Ribhus:

Host	What the approver does
CLI	Rich terminal panel: Approve / Edit / Cancel
Dashboard	Creates pending_approval on the task; UI buttons resolve it
WhatsApp listen	Posts draft to Note to Self; you reply approve / edit: ... / cancel

Same rulebook, three doorbells.

Audit log

Every meaningful allow/deny/execute can be appended to .ribhus/audit.jsonl with secrets redacted.
File: src/ribhus/policy/audit.py

Why it exists: later you can answer "did Ribhus really send that email?" with evidence.

Part 6 - Memory: how Ribhus "remembers you"

Without memory, every run starts as a stranger.

Package: src/ribhus/memory/

Three kinds of memory (student definitions)

1. Short-term - "what we just talked about" (last turn summary).
2. Episodic - "past tasks and outcomes" (like a diary of jobs done).
3. Semantic - "stable facts about the user" (language=Hinglish, city=Bangalore, football club=...).

How it enters a run

Before the model thinks, MemoryService.prompt_addon(prompt) builds a text block that gets glued onto the system prompt.

After a run finishes, the agent can:

- save an episode
- update short-term
- extract prefs from the text ("I live in Pune") via memory/extract.py

Shared vs per-bot memory

With multiple bots:

- some prefs are shared (your city should not fork per bot)
- other notes can be scoped to one bot via bot_id

Pinned keys (language, city, ...) stay shared. That logic is in MemoryService._scope_for_key.

Storage

SQLite file: .ribhus/memory.sqlite

You can also manage it from CLI: ribhus memory list|add|forget|export|disable.

Part 7 - Skills: reusable playbooks

A skill is not magic. It is a folder of instructions that get pasted into the prompt when your request matches.

Example: company research skill says "look for MCA/CIN, cite sources, don't invent funding rounds..."

Anatomy of a skill folder

```
my-skill/
  skill.md      ? title, triggers, instructions (required)
  tools.py      ? optional extra tools for that skill
  examples.md
  tests/
```

How matching works

SkillRegistry.match(prompt) looks for trigger strings in your prompt (simple contains-match, not ML classification).

If matched, prompt_addon injects:

```
## Active skills (company-research)
Follow these skill instructions...
<skill body>
```

Files: skills/registry.py, skills/loader.py, skills/installer.py

Install path

```
ribhus skill install company-research
```

Copies the package into .ribhus/skills/ so your workspace carries it.

Bundled India skills today: company-research, travel, football.

Student takeaway: skills change behavior by changing instructions, not by forking the agent loop.

Part 8 - Channels: WhatsApp Web & Gmail Web

The big misunderstanding to avoid

Ribhus does not call WhatsApp's official Business API.

Ribhus does not call Gmail's official API for the channel tools.

It opens a real Chromium browser, loads web.whatsapp.com / mail.google.com, and automates the UI with Playwright - like a careful robot using the site the way you would.

That is powerful and fragile. Sites change. Accounts can be limited. Docs warn about Terms of Service. You are responsible for your account.

Shared architecture (learn once, apply twice)

For each channel:

Module	Job in plain English
session.py	Start browser with a saved profile folder; detect login / QR
navigator.py	Find the right chat or email thread by name
reader.py	Scrape recent messages / email body
sender.py	Type text and click send
policy.py	Kill switch file, rate limits, "no bulk"
tools.py	Wrap the above as agent Tools
selectors.py	CSS/selectors that match the website DOM

WhatsApp special: Note to Self inbox

File: channels/whatsapp_web/inbox.py

You can run:

```
ribhus whatsapp listen
```

Then Ribhus watches your Note-to-Self chat. You can:

- ask questions there, or
- approve a pending send by typing approve / cancel / edit: new text

That exists so you can approve sends from your phone without sitting at the CLI.

Kill switch

If something goes wrong:

```
ribhus whatsapp kill
# creates .ribhus/whatsapp.killed
```

While that file exists, sends are blocked. resume removes it.

Same pattern for email.

Part 9 - Bots: multiple teammates on one computer

Until bots existed, Ribhus was "one agent."
Now you can create named specialists: Researcher, Writer, Ops...

What is a BotRecord?

File: bots/store.py

A bot has:

- name (what you @mention)
- job (role instructions appended to the system prompt)
- color, label
- status: idle / working / waiting
- is_primary (default bot)

Saved in .ribhus/bots.json.

Chats

File: bots/chat_store.py

Two kinds:

1. Direct - you ? one bot
2. Group - you ? 2-6 bots (with @Name mentions)

Messages are stored in .ribhus/chats.json.

Queues (why bots don't trip over each other)

File: bots/queue.py

Each bot has its own FIFO queue.

- Bot A and Bot B can run at the same time (parallel).
- Two jobs for Bot A run one after another (serial per bot).
- Your new user message can jump ahead of low-priority handoff backlog.

```
Bot A queue: [user msg] [handoff] [fyi]
Bot B queue: [user msg]
             +-- both bots work concurrently
```

Orchestrator

File: bots/runner.py - class BotOrchestrator

This is the dashboard's conductor:

1. User posts to a chat API
2. Orchestrator creates a task (for UI progress/approvals)
3. Enqueues a BotJob
4. Worker builds model + tools + memory(bot_id=...) + Agent
5. Writes the final answer back into the chat
6. Emits typed events on the task (turn.started, tool.finished, ...)

Handoffs with message_bot

When Researcher finishes gathering links, it can call:

```
message_bot(to="Writer", body="Write a 5-bullet brief from these sources: ...", priority="action")
```

That:

1. shows a handoff card in the chats
2. wakes Writer's queue
3. does not block Researcher waiting for Writer (async)

Hop limit stops infinite ping-pong (MAX_HANDOFF_HOPS = 3).

Analogy: Slack DMs between coworkers, not a synchronous function call.

Part 10 - Dashboard: how the website talks to Python

There are two programs when you use the UI:

1. Backend - FastAPI in src/ribhus/web/ (serves /api/...)
2. Frontend - React app in web/src/ (buttons and pages)

ribhus dashboard starts the backend and serves the built SPA (or you run Vite in dev).

Auth (simple but important)

File: web/auth.py

When the server starts, it creates a random session token.
The HTML gets that token injected.
API calls send Authorization: Bearer ....

Also: when bound to localhost, requests with weird Host headers are rejected (LoopbackHostMiddleware). This is a local tool, not a public multi-tenant SaaS.

TaskStore: the dashboard's "run record"

File: web/task_store.py

Every agent run the UI cares about becomes a task with:

- prompt, status (queued -> running -> awaiting_approval -> completed/failed/cancelled)
- events (typed log with seq)
- pending approval payload
- optional bot_id / chat_id

Typed events (why they exist)

Instead of only storing a final answer, Ribhus records a timeline:

```
1 turn.started
2 tool.started   web_search
3 tool.finished  web_search
4 approval.requested  whatsapp_send
5 approval.resolved   approve
6 turn.completed
```

API for polling new events:

```
GET /api/tasks/{id}/events?after_seq=3
```

Student meaning: the UI can refresh "what happened since last time" without re-downloading the whole universe.

Main API groups

URL prefix	Student meaning
/api/tasks	Start work, follow up, approve, read events
/api/bots	Create/rename bots
/api/chats	DMs, groups, post messages
/api/jobs	Job-hunt resume pipeline
/api/skills	Install/list skills
/api/memory	Browse memories
/api/channels	WA/email status + kill/resume
/api/mcp	External servers
/api/config	Safe settings view / edit .env

Frontend map

Page	What you do there
Home	Start a prompt
Chat / New bot / New group	Talk to bots
Tasks / Task detail	Inspect a run + approvals
Job hunt	Tailor resume + send
Skills / Memory / Channels / MCP / Config	Operate the platform

web/src/lib/api.ts is the typed client the React pages call.

Part 11 - CLI: same agent, different doorway

File: cli/main.py

When you run:

```
uv run ribhus "Find hotels in Goa under 8000"
```

the CLI:

1. loads Settings
2. builds PolicyEngine with terminal approver
3. starts browser/WhatsApp/Gmail sessions as needed
4. builds default_registry
5. loads skills + memory
6. calls Agent.run(...)
7. prints the answer with Rich

Subcommands (skill, memory, whatsapp, dashboard, ...) are Typer command groups.

dispatch.py helps decide: "is this argv a subcommand or a natural language prompt?"

Student takeaway: CLI and dashboard are skins. The organ is still Agent + tools + policy.

Part 12 - Job hunt (a specialized workflow)

Job hunt is not a separate agent. It is a guided pipeline that uses:

- LLM to tailor LaTeX
- Tectonic to compile PDF
- Gmail Web channel to send after you approve

Package: src/ribhus/jobs/

Service class: JobHuntService

Steps a student should recite:

1. Save base resume .tex
2. Connect Gmail (same profile folder as email channel)
3. Paste job URL or description
4. Extract keywords / contact email
5. Tailor resume (without inventing fake jobs)
6. Compile PDF
7. Review draft email + PDF
8. Approve -> send

Why separate from free-form chat? Because the UX needs forms, PDF preview, and batching - not only a chat bubble.

Part 13 - MCP: borrowing tools from other programs

MCP = Model Context Protocol.

Plain English: other programs can expose tools over a standard protocol. Ribhus can start those programs (stdio) and wrap each tool as server__tool.

Files: mcp/config.py, mcp/manager.py, mcp/tool.py

Config: .ribhus/mcp.json

Example mental model:

```
Ribhus Agent
   +- calls github__list_issues
```

```
        +- McpProxyTool
             +- MCP GitHub server process
                  +- GitHub API
```

Tokens for those servers stay in env/config - the model should see tool results, not raw secrets (still be careful).

Part 14 - Eval: how we test without burning API money

Package: src/ribhus/eval/

ribhus eval runs a catalog of cases with fake models and mocked HTTP when possible.

Categories include: basic tools, safety (send cannot auto-approve), India skill expectations, WhatsApp fixtures, dashboard smoke, etc.

Student meaning: before you change policy code, run evals so you don't accidentally make email_send auto-approve.

Part 15 - Full walkthrough #1 (CLI research)

Command:

```
uv run ribhus "Research this Indian company: Zoho"
```

What happens inside, slowly:

1. CLI parses the string as a prompt (not a subcommand).
2. Settings load API key + model.
3. SkillRegistry sees trigger words -> injects company-research instructions.
4. Memory may inject "user prefers Hinglish" if stored.
5. Agent.run starts.
6. Model requests web_search.
7. Registry executes search (low risk -> no popup).
8. Model requests web_extract on a promising URL.
9. Model writes a sourced brief.
10. Memory stores an episode.
11. CLI prints the brief.

No WhatsApp, no approval UI - because nothing high-risk was required.

Part 16 - Full walkthrough #2 (Dashboard WhatsApp send)

1. You open the dashboard (ribhus dashboard).
2. You open bot "Ops" chat.
3. You type: "WhatsApp Rahul that dinner is at 8."
4. API POST /api/chats/{id}/messages -> BotOrchestrator.enqueue_user_message.
5. A task is created (status=queued then running). Event: turn.started.
6. Agent decides channel is WhatsApp (heuristics + system prompt).
7. Model calls whatsapp_send.
8. Policy sees force_approval -> dashboard approver creates pending_approval.

9. Event: approval.requested. Bot status may show waiting.
10. UI shows draft + Approve / Edit / Deny.
11. You click Approve -> approval.resolved.
12. Sender types into WhatsApp Web and sends.
13. Event: tool.finished, then turn.completed.
14. Chat shows the assistant summary.

If you Deny, the tool result tells the model "user cancelled," and it should not pretend the message was sent.

Part 17 - Full walkthrough #3 (Group handoff)

1. Create bots Researcher and Writer; put them in a group.
2. You say: @Researcher shortlist 3 Goa hotels under 8k.
3. Only Researcher is scheduled (mention).
4. Researcher uses search/browser tools, finishes.
5. Researcher calls message_bot to Writer with notes.
6. Event handoff.sent on Researcher's task; Writer's queue gets a handoff job.
7. Writer writes the polished shortlist into chat (and maybe back into the sender chat for action handoffs).

You did not manually copy-paste between bots. The platform did.

Part 18 - How pieces call each other (dependency direction)

Read this as gravity: arrows point toward "more fundamental" layers.

```
CLI / React UI / WhatsApp inbox
        |
        v
FastAPI routers  or  Typer commands
        |
        v
BotOrchestrator (optional) --? TaskStore / ChatStore / BotStore
        |
        v
Agent  --uses--?  OpenAICompatibleModel
   |
   +--uses--? SkillRegistry (prompt text only)
   +--uses--? MemoryService (prompt text + persistence)
   +--uses--? ToolRegistry
                |
                +-- PolicyEngine (+ Approver UI)
                +-- AuditLog
                +-- individual Tools
                       +-- Playwright sessions (browser/WA/Gmail)
                       +-- filesystem / shell / python
                       +-- MCP proxy tools
```

If you add a feature, ask: "Which layer does it belong to?"

- New capability the model should choose -> Tool
- New standing workflow instructions -> Skill
- New website login automation -> Channel
- New multi-agent routing -> bots/
- New screen -> web UI + maybe router

Part 19 - Config knobs students actually touch

From .env / Settings:

Knob	Plain meaning
RIBHUS_API_KEY	Key for the model provider
RIBHUS_BASE_URL	Provider URL (OpenAI / OpenRouter / Ollama)
RIBHUS_MODEL	Model id
RIBHUS_AUTO_APPROVE	Skip normal approvals (NOT sends with force_approval)
RIBHUS_APPROVE_AT	Risk threshold to start asking
RIBHUS_BROWSER_HEADED	Show Chromium window
RIBHUS_WHATSAPP_HEADED	Needed for QR login
RIBHUS_MEMORY_ENABLED	Turn memory on/off
RIBHUS_MCP_ENABLED	Load MCP servers

Part 20 - What Ribhus is not (clear boundaries)

Say these out loud so you don't confuse yourself later:

1. Not a fine-tuned India LLM you host yourself (you bring a model).
2. Not official WhatsApp/Gmail APIs.
3. Not a cloud microVM "computer in the sky" (everything local unless you point tools at the internet).
4. Not finished Grok Bot parity (Stop, Always-allow fingerprints, routines, vault - see docs/blueprint-gap-tasks.md).
5. Not allowed to silently send mail/WhatsApp without a human gate.

Part 21 - Study plan (if you want to learn the code in order)

Spend one session per step. Do not skip.

1. Read tools/base.py + one simple tool (web_search.py).
2. Read policy/engine.py.
3. Read Agent.run in agent/loop.py top to bottom once.
4. Run uv run ribhus "What is 17*19?" and watch it use python or answer directly.
5. Read memory/service.py prompt_addon + record_episode.
6. Read one skill's skill.md.
7. Read bots/queue.py + message_tool.py.
8. Read web/routers/chats.py + bots/runner.py enqueue path.
9. Read WhatsApp policy.py + tools.py send path.
10. Skim jobs/service.py prepare/send.

When you finish that list, you "own" the architecture.

Part 22 - Glossary (exam cheat sheet)

Word	One-line definition
Agent	Loop that calls the model and tools until done
Tool	Named capability the model can invoke
Policy	Rules that decide if a tool needs human OK
Approver	UI/function that collects Approve/Edit/Cancel
Skill	Packaged instructions injected into the prompt
Memory	Local facts/episodes stored in SQLite
Channel	Browser automation for WhatsApp/Gmail
Bot	Named durable agent identity + queue
Handoff	Async message from one bot to another
Task	Dashboard record of one run + events
MCP	Standard way to plug in external tools
Workspace	Project folder file tools are allowed to touch
.ribhus/	Hidden folder of local runtime state

Closing

Ribhus is a local office for an LLM intern:

- the intern thinks (model)
- the office supplies equipment (tools)
- the office enforces safety (policy)
- the office keeps notes (memory)
- the office has playbooks (skills)
- the office can hire multiple named interns (bots)
- you can talk through a counter (CLI), a window (dashboard), or a phone note (WhatsApp listen)

If something feels "vague" again, pick one walkthrough (Parts 15-17) and trace it in the debugger or with print/logging. Architecture becomes real when you watch one request move.

If this guide and the code disagree, trust the code - then update the guide.